

C++ 프로그래밍

(입출력과 파일처리)

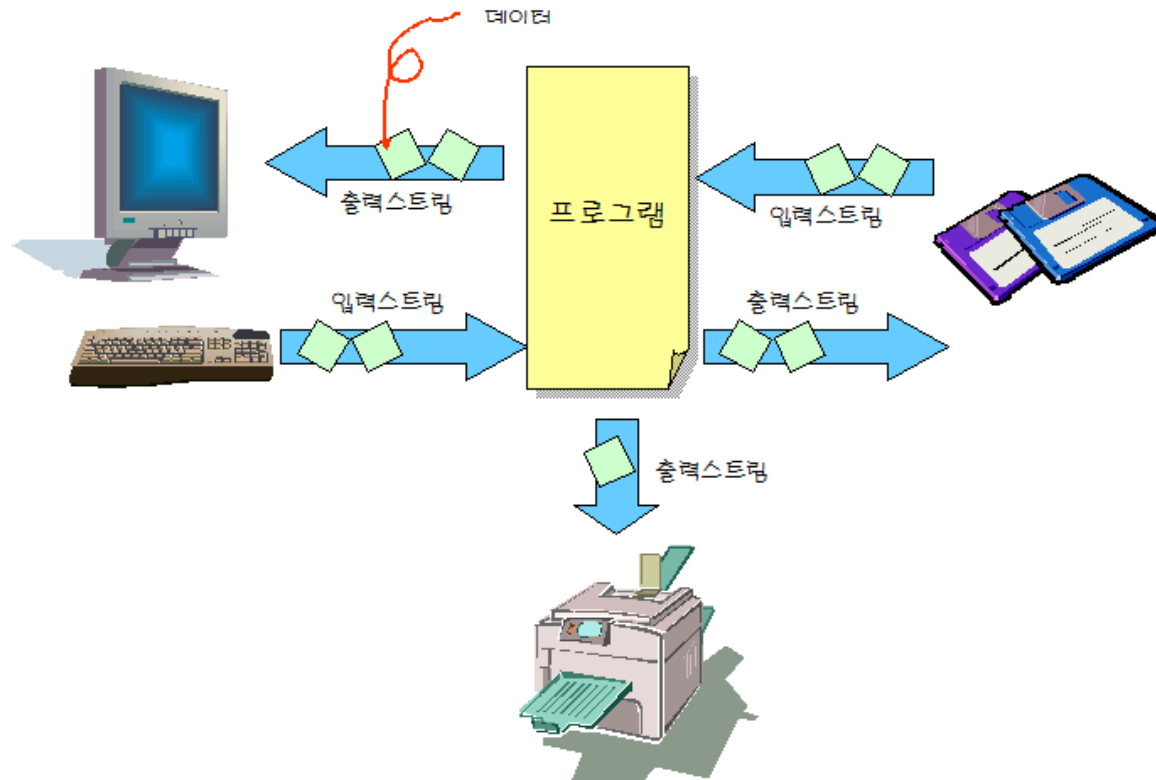
12주차

목차

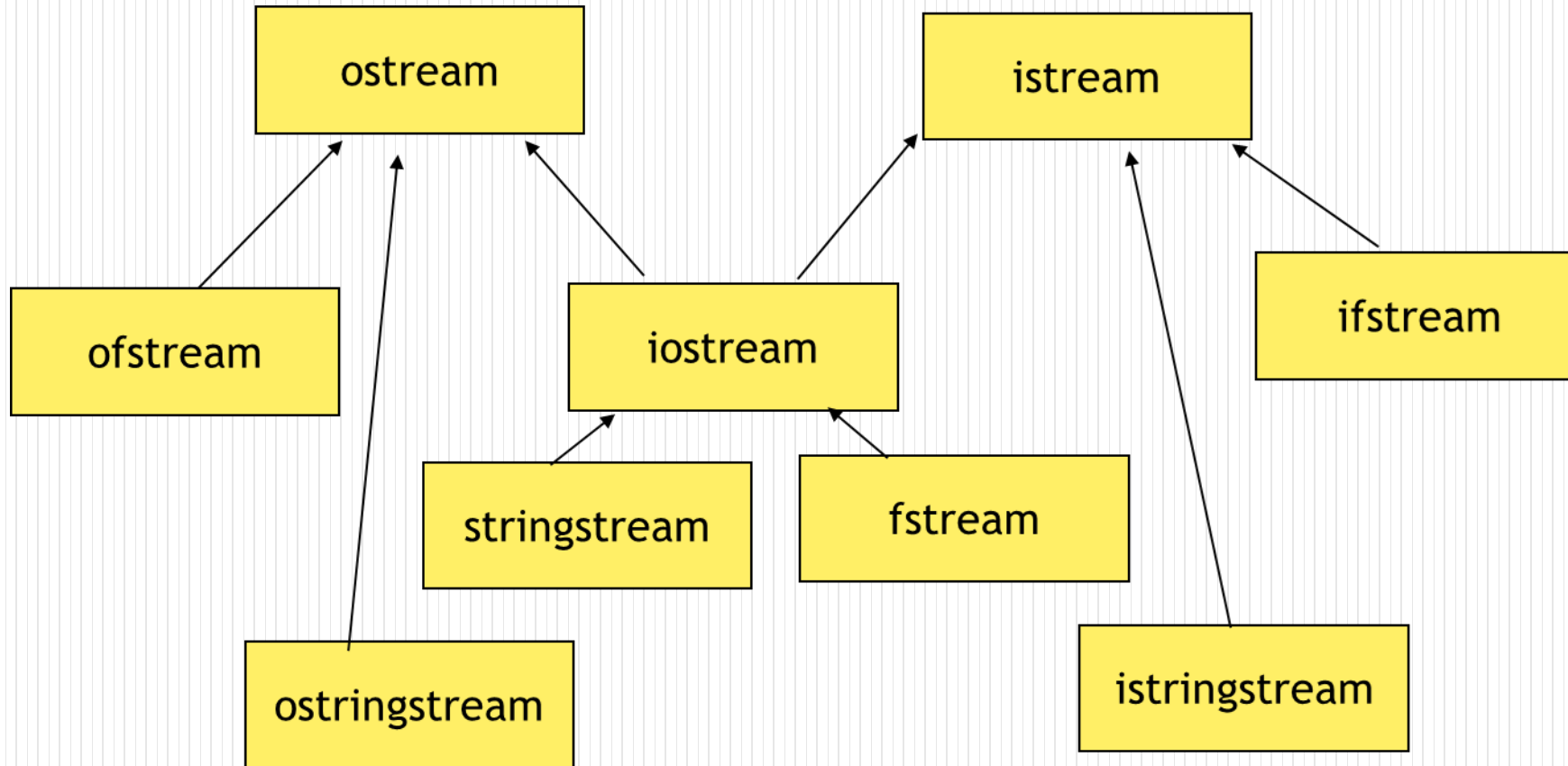
- 파일 입출력
- 텍스트 파일과 이진 파일
- 순차 파일과 랜덤 파일

파일 입출력

- 스트림(stream)
 - 스트림(stream)은 “순서가 있는 데이터의 연속적인 흐름” 이다.
 - 스트림은 입출력을 물의 흐름처럼 간주하는 것이다.

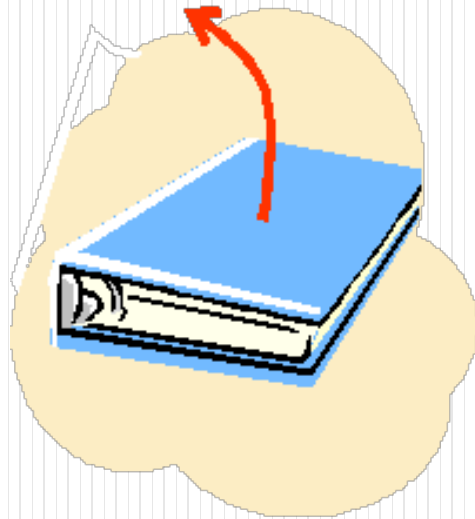


파일 입출력 (계속)

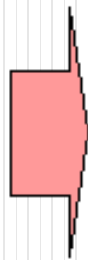


파일 입출력 (계속)

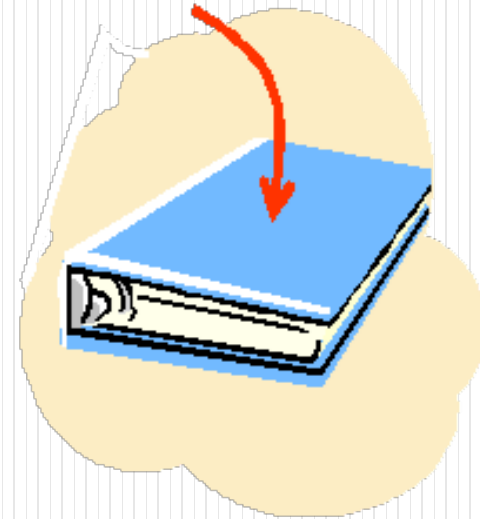
- 파일 처리의 순서
 - 파일을 다룰 때는 반드시 다음과 같은 순서를 지켜야 한다.



파일 열기



파일 읽기와 쓰기



파일 닫기

파일 입출력 (계속)

- "<<" 과 ">>"을 이용한 입출력

- 입력

```
ifstream is;  
is.open("test.txt");  
int number;  
is >> number;
```

- 출력

```
ofstream os;  
os.open("result.txt");  
os<<number;
```

파일 입출력 (계속)

- 예제
 - 학생 데이터를 파일에 저장하고 읽어보자

```
int number;    // 학번  
char name[30]; // 이름  
int score;     // 성적
```



파일 입출력 (계속)

- 예제 (계속)

```
#include <iostream>
#include <fstream>           // 파일 입출력을 하려면 헤더 파일을 포함하여야 한다.
using namespace std;
```

```
int main()
{
    ifstream is;
    is.open("score.txt");
    if( !is ) {                // ! 연산자 오버로딩
        cerr << "파일 오픈에 실패하였습니다" << endl;
        exit( 1 );
    }
```

```
    int number;                // 학번
    char name[30];             // 이름
    int score;                  // 성적
    is >> number >> name >> score;
```

```
    ofstream os;
    os.open("result.txt");
    os << number << " " << name << " " << score << endl;
```

```
    is.close();
    os.close();
    return 0;
```

```
}
```

score.txt

```
20100001 홍길동 100
20100002 김유신 90
20100003 강감찬 80
```

result.txt

```
20100001 홍길동 100
```

파일 입출력 (계속)

- 키보드에서 받은 문장 저장 예제

```
#include <iostream>
#include <fstream>           // 파일 입출력을 하려면 헤더 파일을 포함하여야 한다.
using namespace std;

int main()
{
    ofstream os;
    char c;
    os.open("test.txt");
    while( cin.get(c) )
    {
        os.put(c);
    }
    os.close();
    return 0;
}
```

파일 test.txt

This is a test.

실행 결과

This is a test.

^Z

Press any key to continue

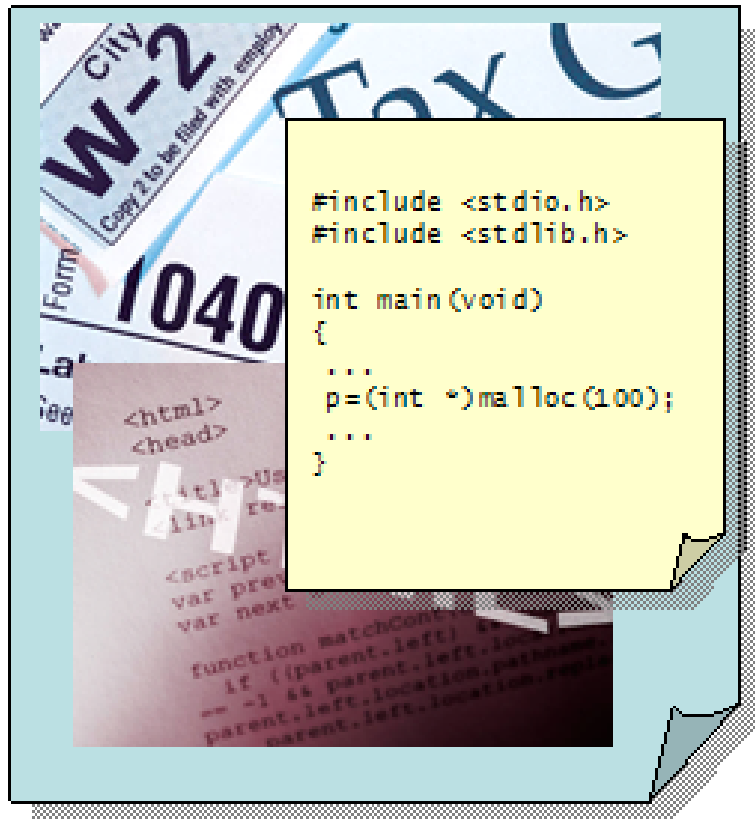
파일 입출력 (계속)

- 출력 형식 지정

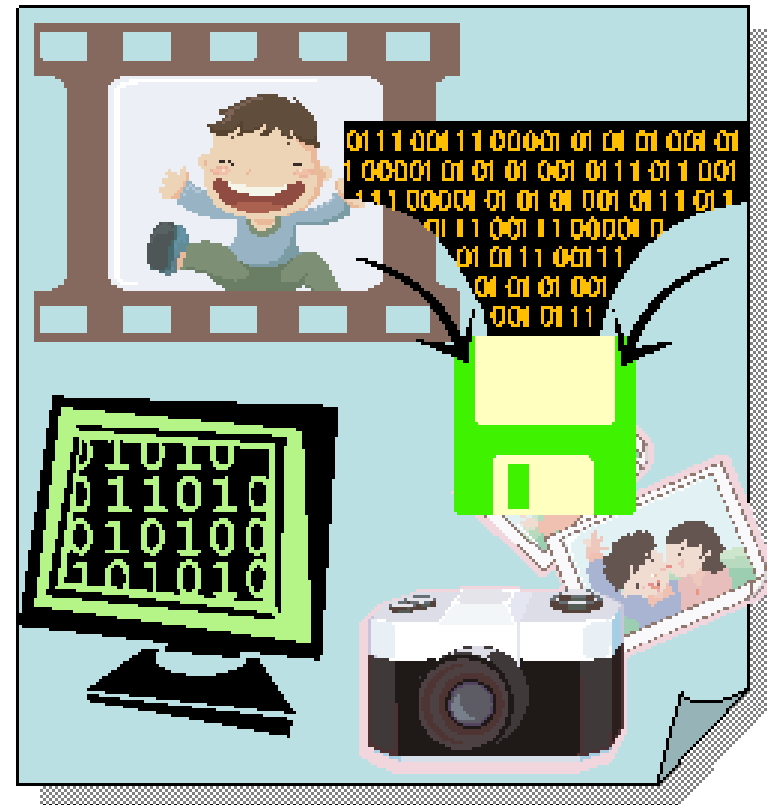
플래그	설명
<code>ios::fixed</code>	고정 소수점 표기법 사용
<code>ios::scientific</code>	과학적 표기법 사용(지수를 이용하여 표기)
<code>ios::showpoint</code>	소수점을 반드시 표시한다.
<code>ios::showpos</code>	양수 부호를 반드시 출력한다.
<code>ios::right</code>	값을 출력할 때 오른쪽 정렬을 사용한다.
<code>ios::left</code>	값을 출력할 때 왼쪽 정렬을 사용한다.
<code>ios::dec</code>	값을 출력할 때 10진법을 사용한다.
<code>ios::oct</code>	값을 출력할 때 8진법을 사용한다.
<code>ios::hex</code>	값을 출력할 때 16진법을 사용한다.
<code>ios::uppercase</code>	지수나 16진법으로 표시할 때 대문자를 사용한다.
<code>ios::show</code>	8진수이면 앞에 0을 붙이고 16진수이면 앞에 0x를 붙인다.

```
cout.precision(3);    // 소수점 이하 지리수를 3자리로 설정
cout.setf(ios::fixed);
cout.setf(ios::showpoint);
cout.setf(ios::left);
cout.setf(ios::left | ios::showpoint);
cout.unsetf(ios::uppercase);    // 설정 해제
```

텍스트와 이진 파일



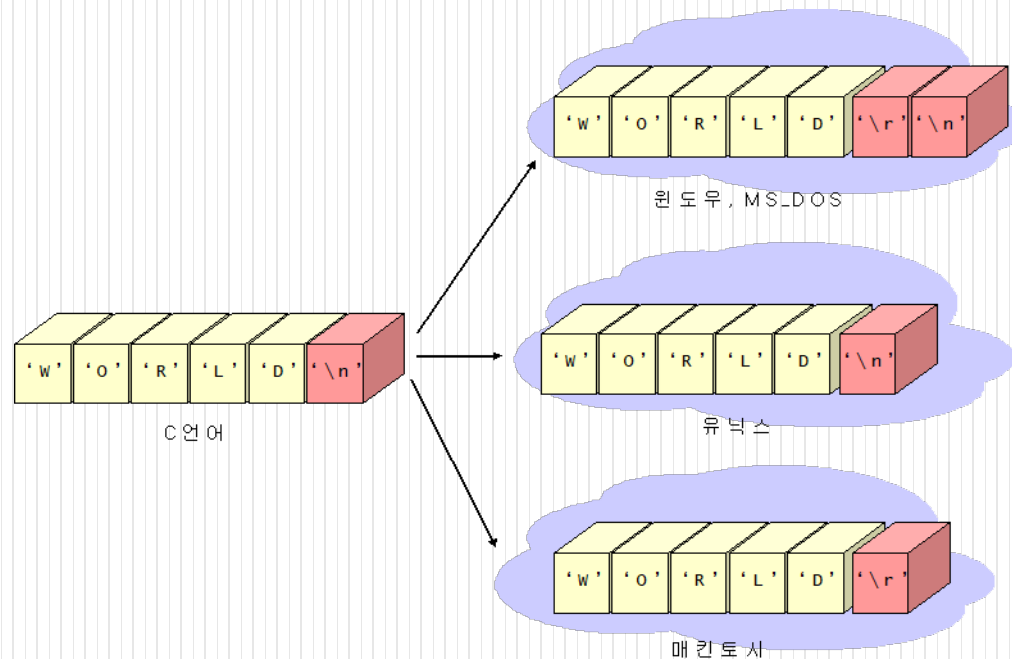
텍스트 파일: 문자로 구성된 파일



이진 파일: 데이터로 구성된 파일

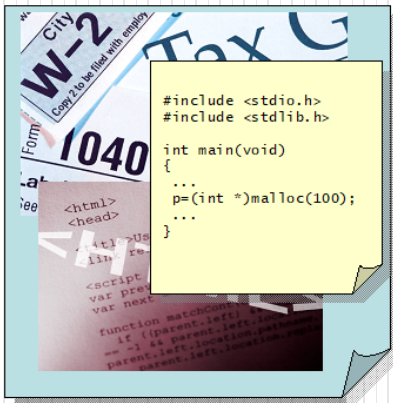
텍스트와 이진 파일 (계속)

- 텍스트 파일(text file)
 - 텍스트 파일은 사람이 읽을 수 있는 텍스트가 들어 있는 파일
 - (예) c프로그램 소스 파일이나 메모장 파일
 - 텍스트 파일은 아스키 코드를 이용하여 저장
 - 텍스트 파일은 연속적인 라인들로 구성

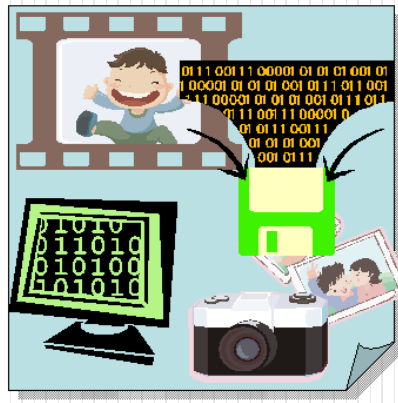


텍스트와 이진 파일 (계속)

- 이진 파일(binary file)
 - 이진 파일은 사람이 읽을 수는 없으나 컴퓨터는 읽을 수 있는 파일
 - 이진 데이터가 직접 저장되어 있는 파일
 - 이진 파일은 텍스트 파일과는 달리 라인들로 분리되지 않는다.
 - 모든 데이터들은 문자열로 변환되지 않고 입출력
 - 이진 파일은 특정 프로그램에 의해서만 판독이 가능
 - (예) c프로그램 소스 파일, 사운드 파일, 이미지 파일



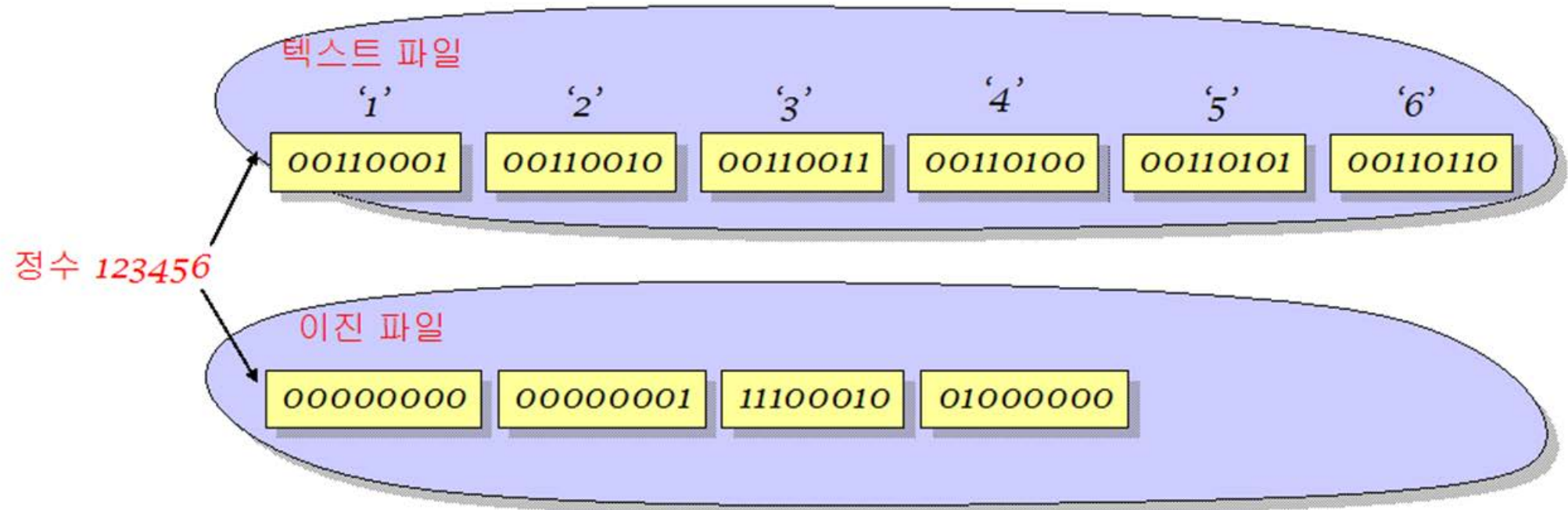
텍스트 파일: 문자로 구성된 파일



이진 파일: 데이터로 구성된 파일

텍스트와 이진 파일 (계속)

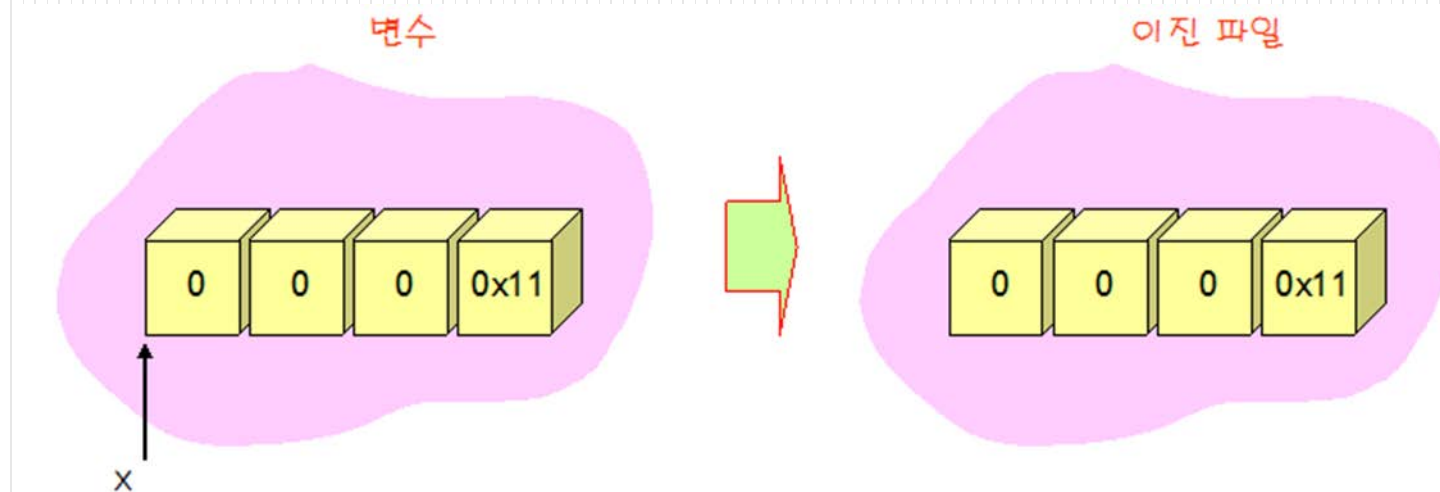
- 텍스트와 이진 파일의 저장 방법 비교



텍스트와 이진 파일 (계속)

- 이진 파일 입출력

```
ofstream os;  
os.open("test.dat", ofstream::binary);  
int x = 5;  
os.write((char*)&x, sizeof(int));
```



텍스트와 이진 파일 (계속)

- 예제

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main()
```

```
{
```

```
    int buffer[] = { 10, 20, 30, 40, 50 };
```

```
    ofstream os;
```

```
    os.open("test.dat",ofstream::binary);
```

```
    if( os.fail() )
```

```
    {
```

```
        cout << "binary.txt 파일을 열 수 없습니다." << endl;
```

```
        exit(1);
```

```
    }
```

```
    os.write((char *)&buffer, sizeof(buffer));
```

```
    os.close();
```

```
    return 0;
```

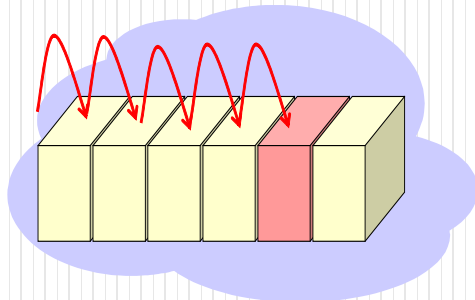
```
}
```

파일 test.dat

```
00000000: 0A 00 00 00 14 00 00 00|1E 00 00 00 28 00 00 00 | 00 00 00 00 00 00 00 00
00000010: 32 00 00 00          |          | 00 00 00 00 00 00 00 00
```

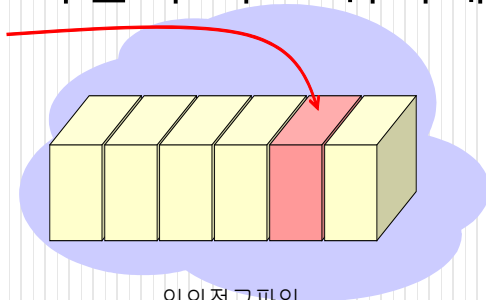
순차 파일과 랜덤 파일

- 임의 접근 파일
 - 순차 접근(sequential access)방법
 - 데이터를 파일의 처음부터 순차적으로 읽거나 기록하는 방법



순차접근파일

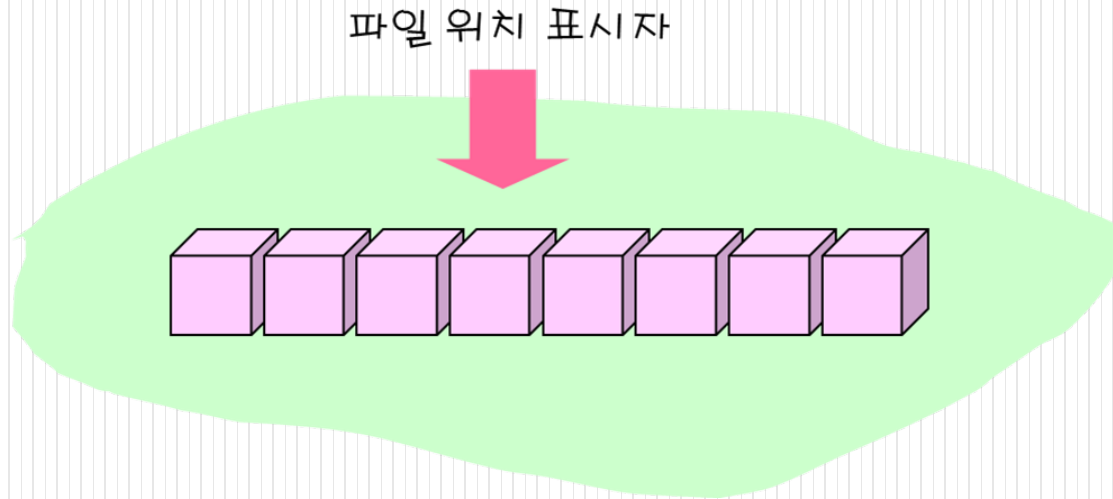
- 임의 접근(random access)방법
 - 파일의 어느 위치에서든지 읽기와 쓰기가 가능한 방법



임의접근파일

순차 파일과 랜덤 파일 (계속)

- 임의 접근 파일의 원리
 - 파일 위치 표시자
 - 읽기와 쓰기 동작이 현재 어떤 위치에서 이루어지는지를 나타낸다.

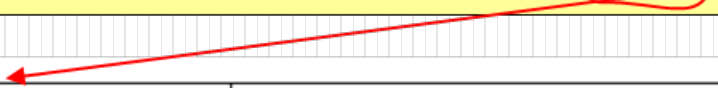


- 강제로 파일 위치 표시자를 이동시키면 임의 접근이 가능

순차 파일과 랜덤 파일 (계속)

- 임의 접근 관련 함수

`seekg(long offset, seekdir way);`



way	설명
<code>ios::beg</code>	처음부터의 offset
<code>ios::cur</code>	현재 위치부터의 offset
<code>ios_end</code>	파일의 끝에서부터의 offset

```
is.seekg(ios::beg, 100);
```

```
is.seekg(ios::end, 0);
```

```
is.seekg(ios::cur, -100);
```

순차 파일과 랜덤 파일 (계속)

● 예제

```
const int SIZE=1000;
void init_table(int table[], int size);

int main()
{
    int table[SIZE];
    int data;
    long pos;

    // 배열을 초기화한다.
    init_table(table, SIZE);

    // 이진 파일을 쓰기 모드로 연다.
    ofstream os;
    os.open("test.dat",ofstream::binary);
    if( os.fail() )
    {
        cout << "test.dat 파일을 열 수 없습니다." << endl;
        exit(1);
    }
}
```

```
// 배열을 이진 모드로 파일에 저장한다.
os.write((char *)table, sizeof(table));
os.close();

// 이진 파일을 읽기 모드로 연다.
ifstream is;
is.open("test.dat",ifstream::binary);
if( is.fail() )
{
    cout << "test.dat 파일을 열 수 없습니다." << endl;
    exit(1);
}
```

```
// 사용자가 선택한 위치의 정수를 파일로부터 읽는다.
while(1)
{
    cout << "파일에서의 위치를 입력하십시오(0에서 999, 종료-1): ";
    cin >> pos;
    if( pos == -1 ) break;
    is.seekg(pos*sizeof(int), ios::beg);
    is.read((char *)&data, sizeof(int));
    cout << pos << " 위치의 값은" << data << "입니다." << endl;
}
is.close();
return 0;
}

// 배열을 인덱스의 제공으로 채운다.
void init_table(int table[], int size)
{
    int i;

    for(i = 0; i < size; i++)
        table[i] = i * i;
}
```

파일에서의 위치를 입력하십시오(0에서 999, 종료 -1): 3
3 위치의 값은 9입니다.
파일에서의 위치를 입력하십시오(0에서 999, 종료 -1): 9
9 위치의 값은 81입니다.
파일에서의 위치를 입력하십시오(0에서 999, 종료 -1): -1